

Day 5 — 告警联动与上位机（完整代码版）

部分内容由豆包生成

Day 5 — 告警联动与上位机（完整代码版）



今日目标：用手捂住 SHT30 升温超限 → 蜂鸣器响、OLED 告警、继电器吸合、MQTT 收到告警消息；松开后温度回落 → 自动恢复。PyQt5 上位机能显示实时数据并远程控制继电器。

开发顺序：阈值告警逻辑 → 告警联动 → 恢复与迟滞 → 按键设阈值 → Flash保存 → 数据滤波 → PyQt5上位机。

第一步：阈值告警逻辑完善

Day3 已经有了基础的阈值检测，这里完善为三路独立阈值 + 迟滞区间 + 告警等级。

新建 APP/alarm_manager.h 和 alarm_manager.c。

代码块

```
1  #ifndef __ALARM_MANAGER_H
2  #define __ALARM_MANAGER_H
3
4  #include "appdata.h"
5
6  /* 告警等级 */
7  #define ALARM_NONE      0
8  #define ALARM_WARNING  1  /* 注意：超过阈值的 80% */
9  #define ALARM_CRITICAL  2  /* 异常：超过阈值 */
10
11 /* 告警源位图 */
12 #define ALARM_SRC_TEMP  (1 << 0)
13 #define ALARM_SRC_VIB   (1 << 1)
14 #define ALARM_SRC_CURR  (1 << 2)
15
16 typedef struct {
17     uint8_t  active;          /* 是否有告警 */
18     uint8_t  level;          /* 当前最高告警等级 */
19     uint8_t  source_mask;     /* 告警源位图 */
20     uint32_t start_time;      /* 告警开始时间 (tick) */
21     uint32_t duration;        /* 持续时间 (ms) */
22 } AlarmStatus_t;
```

```

23
24 void ALARM_Init(void);
25 void ALARM_Update(const SensorData_t *data);
26 uint8_t ALARM_IsActive(void);
27 uint8_t ALARM_GetLevel(void);
28 uint8_t ALARM_GetSourceMask(void);
29 void ALARM_GetStatus(AlarmStatus_t *status);
30 const char* ALARM_GetSourceName(uint8_t source_bit);
31
32 #endif

```

代码块

```

1  #include "alarm_manager.h"
2  #include "esp8266.h"
3  #include "mqtt_publisher.h"
4  #include "relay.h"
5  #include "buzzer.h"
6  #include <stdio.h>
7
8  static AlarmStatus_t alarm_status;
9  static uint8_t alarm_published = 0; /* 防止重复发布告警 */
10
11 /* 迟滞比例: 恢复时数值需低于阈值的 (1-HYSTERESIS) 倍 */
12 #define HYSTERESIS 0.10f /* 10% 迟滞 */
13
14 void ALARM_Init(void)
15 {
16     alarm_status.active = 0;
17     alarm_status.level = ALARM_NONE;
18     alarm_status.source_mask = 0;
19     alarm_status.start_time = 0;
20     alarm_status.duration = 0;
21     alarm_published = 0;
22 }
23
24 void ALARM_Update(const SensorData_t *data)
25 {
26     uint8_t new_mask = 0;
27     uint8_t new_level = ALARM_NONE;
28
29     /* 温度检测 */
30     if (data->temperature > g_threshold.temp_high) {
31         new_mask |= ALARM_SRC_TEMP;
32         new_level = ALARM_CRITICAL;
33     } else if (data->temperature > g_threshold.temp_high * 0.8f) {

```

```

34     new_mask |= ALARM_SRC_TEMP;
35     if (new_level < ALARM_WARNING) new_level = ALARM_WARNING;
36 }
37
38 /* 振动检测 */
39 if (data->vibration > g_threshold.vib_high) {
40     new_mask |= ALARM_SRC_VIB;
41     new_level = ALARM_CRITICAL;
42 } else if (data->vibration > g_threshold.vib_high * 0.8f) {
43     new_mask |= ALARM_SRC_VIB;
44     if (new_level < ALARM_WARNING) new_level = ALARM_WARNING;
45 }
46
47 /* 电流检测 */
48 if (data->current > g_threshold.curr_high) {
49     new_mask |= ALARM_SRC_CURR;
50     new_level = ALARM_CRITICAL;
51 } else if (data->current > g_threshold.curr_high * 0.8f) {
52     new_mask |= ALARM_SRC_CURR;
53     if (new_level < ALARM_WARNING) new_level = ALARM_WARNING;
54 }
55
56 /* 状态机：带迟滞的告警触发/恢复 */
57 if (!alarm_status.active && new_level >= ALARM_CRITICAL) {
58     /* 触发告警 */
59     alarm_status.active = 1;
60     alarm_status.level = new_level;
61     alarm_status.source_mask = new_mask;
62     alarm_status.start_time = xTaskGetTickCount();
63     g_alarm_active = 1;
64     alarm_published = 0;
65     printf("ALARM TRIGGERED: mask=0x%02X level=%d\r\n", new_mask,
new_level);
66 } else if (alarm_status.active) {
67     /* 更新告警等级和源 */
68     alarm_status.level = new_level;
69     alarm_status.source_mask = new_mask;
70     alarm_status.duration = (xTaskGetTickCount() -
alarm_status.start_time);
71
72     /* 恢复条件：所有指标都低于阈值的 (1-HYSTERESIS) 倍 */
73     uint8_t can_recover = 1;
74     if (data->temperature > g_threshold.temp_high * (1.0f - HYSTERESIS))
can_recover = 0;
75     if (data->vibration > g_threshold.vib_high * (1.0f - HYSTERESIS))
can_recover = 0;

```

```

76         if (data->current > g_threshold.curr_high * (1.0f - HYSTERESIS))
            can_recover = 0;
77
78         if (can_recover) {
79             alarm_status.active = 0;
80             alarm_status.level = ALARM_NONE;
81             alarm_status.source_mask = 0;
82             g_alarm_active = 0;
83             alarm_published = 0;
84             printf("ALARM RECOVERED, duration=%ums\r\n",
            alarm_status.duration);
85         }
86     }
87 }
88
89 uint8_t ALARM_IsActive(void) { return alarm_status.active; }
90 uint8_t ALARM_GetLevel(void) { return alarm_status.level; }
91 uint8_t ALARM_GetSourceMask(void) { return alarm_status.source_mask; }
92
93 void ALARM_GetStatus(AlarmStatus_t *status)
94 {
95     *status = alarm_status;
96 }
97
98 const char* ALARM_GetSourceName(uint8_t source_bit)
99 {
100     switch (source_bit) {
101         case ALARM_SRC_TEMP: return "Temperature";
102         case ALARM_SRC_VIB:  return "Vibration";
103         case ALARM_SRC_CURR: return "Current";
104         default: return "Unknown";
105     }
106 }

```

第二步：告警联动完整实现

在 AlarmTask 中调用 ALARM_Update，并执行联动动作。替换 Day3 的 AlarmTask：

代码块

```

1 void AlarmTask(void *pvParameters)
2 {
3     SensorData_t data;
4     uint32_t tick_ms = 0;
5     AlarmEvent_t mqtt_alarm;
6

```

```

7     ALARM_Init();
8
9     for (;;) {
10         /* 读取最新传感器数据 */
11         if (xSemaphoreTake(xDataMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
12             data = g_sensor_data;
13             xSemaphoreGive(xDataMutex);
14
15             /* 更新告警状态 */
16             uint8_t was_active = alarm_status.active;
17             ALARM_Update(&data);
18
19             /* 告警触发瞬间：执行联动 + 发布 MQTT */
20             if (alarm_status.active && !was_active) {
21                 /* 1. 继电器吸合断电 */
22                 RELAY_On();
23                 /* 2. 蜂鸣器急促告警 */
24                 BUZZER_AlarmPattern(1);
25                 /* 3. 发布 MQTT 告警消息 */
26                 if (alarm_status.source_mask & ALARM_SRC_TEMP) {
27                     mqtt_alarm.type = 1;
28                     mqtt_alarm.value = data.temperature;
29                     mqtt_alarm.threshold = g_threshold.temp_high;
30                 } else if (alarm_status.source_mask & ALARM_SRC_VIB) {
31                     mqtt_alarm.type = 2;
32                     mqtt_alarm.value = data.vibration;
33                     mqtt_alarm.threshold = g_threshold.vib_high;
34                 } else {
35                     mqtt_alarm.type = 3;
36                     mqtt_alarm.value = data.current;
37                     mqtt_alarm.threshold = g_threshold.curr_high;
38                 }
39                 mqtt_alarm.level = alarm_status.level;
40                 MQTT_Pub_Alarm(&mqtt_alarm);
41
42                 printf("ALARM ACTION: relay=ON buzzer=ON mqtt=published\r\n");
43             }
44
45             /* 告警恢复瞬间：解除联动 */
46             if (!alarm_status.active && was_active) {
47                 RELAY_Off();
48                 BUZZER_AlarmPattern(0);
49                 printf("ALARM RECOVER: relay=OFF buzzer=OFF\r\n");
50             }
51         }
52
53         /* 更新蜂鸣器节奏 */

```

```

54         tick_ms += 100;
55         BUZZER_Update(tick_ms);
56
57         vTaskDelay(pdMS_TO_TICKS(100));
58     }
59 }

```

同时从 SensorTask 中移除 Day3 写的简易告警检测代码（现在统一由 AlarmTask 处理）。

第三步：OLED 告警显示优化

在 DisplayTask 的 PAGE_REALTIME 中，告警时显示告警信息：

代码块

```

1  /* 在 DisplayTask 的 PAGE_REALTIME case 中，数据显示后添加： */
2  if (g_alarm_active) {
3      /* 第7页显示告警源 */
4      OLED_ShowString(0, 7, "ALARM!");
5      if (alarm_status.source_mask & ALARM_SRC_TEMP) OLED_ShowString(48, 7, "T");
6      if (alarm_status.source_mask & ALARM_SRC_VIB)  OLED_ShowString(54, 7, "V");
7      if (alarm_status.source_mask & ALARM_SRC_CURR) OLED_ShowString(60, 7, "I");
8  } else {
9      OLED_ShowString(0, 7, "System OK");
10 }

```

告警闪烁效果改进——整个屏幕反白闪烁比只反白标题行更醒目：

代码块

```

1  /* 替换 Day3 的告警闪烁代码 */
2  if (g_alarm_active) {
3      blink_tick++;
4      if ((blink_tick % 6) < 3) {
5          /* 奇数周期：正常显示 */
6      } else {
7          /* 偶数周期：全屏幕反白（闪烁告警） */
8          for (int page = 0; page < 8; page++) {
9              for (int col = 0; col < 128; col++) {
10                 OLED_GRAM[page][col] = ~OLED_GRAM[page][col];
11             }
12         }
13     }
14 }

```

第四步：按键设置阈值（完整交互）

Day3 已有基础的阈值加减，这里完善为完整的设置交互流程，包括进入/退出设置模式、选择项高亮、确认保存。

在 `appdata.h` 中添加设置状态：

代码块

```
1  /* 在 appdata.h 中添加 */
2  typedef enum {
3      SET_MODE_EXIT = 0,      /* 正常模式 */
4      SET_MODE_SELECT,      /* 选择要修改的阈值项 */
5      SET_MODE_ADJUST      /* 调整数值 */
6  } SetMode_t;
7
8  extern SetMode_t g_set_mode;
9  extern uint8_t g_set_selected; /* 0=温度,1=振动,2=电流 */
```

在 KeyTask 的按键回调中完善设置逻辑：

代码块

```
1  /* 替换 Day3 的 OnKeyPressed 函数 */
2  void OnKeyPressed(uint8_t key)
3  {
4      printf("Key: 0x%02X\r\n", key);
5
6      switch (g_set_mode) {
7          case SET_MODE_EXIT:
8              /* 正常模式下的按键功能 */
9              switch (key) {
10                 case KEY_A:
11                     g_current_page = (g_current_page + 1) % PAGE_MAX;
12                     break;
13                 case KEY_B:
14                     RELAY_Toggle();
15                     break;
16                 case KEY_C:
17                     BUZZER_Beep(100);
18                     break;
19                 case KEY_STAR:
20                     /* 长按 * 进入设置模式（这里简化为单击进入） */
21                     g_set_mode = SET_MODE_SELECT;
22                     g_set_selected = 0;
23                     g_current_page = PAGE_THRESHOLD;
24                     printf("Enter threshold settings\r\n");
```

```

25         break;
26     default:
27         break;
28     }
29     break;
30
31     case SET_MODE_SELECT:
32         /* 选择阈值项模式 */
33         switch (key) {
34             case KEY_2: /* 上移 */
35                 g_set_selected = (g_set_selected + 2) % 3;
36                 break;
37             case KEY_8: /* 下移 */
38                 g_set_selected = (g_set_selected + 1) % 3;
39                 break;
40             case KEY_A: /* 确认进入调整 */
41                 g_set_mode = SET_MODE_ADJUST;
42                 break;
43             case KEY_STAR: /* 退出设置 */
44                 g_set_mode = SET_MODE_EXIT;
45                 g_current_page = PAGE_REALTIME;
46                 printf("Exit settings\r\n");
47                 break;
48             default:
49                 break;
50         }
51         break;
52
53     case SET_MODE_ADJUST:
54         /* 调整数值模式 */
55         switch (key) {
56             case KEY_2: /* 加 */
57             case KEY_POUND:
58                 switch (g_set_selected) {
59                     case 0: g_threshold.temp_high += 1.0f; break;
60                     case 1: g_threshold.vib_high += 0.1f; break;
61                     case 2: g_threshold.curr_high += 0.1f; break;
62                 }
63                 break;
64             case KEY_8: /* 减 */
65             case KEY_STAR:
66                 switch (g_set_selected) {
67                     case 0:
68                         if (g_threshold.temp_high > 10.0f)
69                             g_threshold.temp_high -= 1.0f;
70                         break;
71                     case 1:

```

```

72             if (g_threshold.vib_high > 0.2f)
73                 g_threshold.vib_high -= 0.1f;
74             break;
75         case 2:
76             if (g_threshold.curr_high > 0.2f)
77                 g_threshold.curr_high -= 0.1f;
78             break;
79     }
80     break;
81     case KEY_A: /* 确认保存 */
82         /* 保存到 Flash */
83         /* FLASH_SaveThreshold(&g_threshold); */
84         g_set_mode = SET_MODE_SELECT;
85         printf("Threshold saved\r\n");
86         break;
87     case KEY_B: /* 取消返回 */
88         g_set_mode = SET_MODE_SELECT;
89         break;
90     default:
91         break;
92 }
93 break;
94 }
95 }

```

在 DisplayTask 的 PAGE_THRESHOLD 中，高亮当前选中项：

代码块

```

1  /* 在 PAGE_THRESHOLD 中，替换原来的静态显示 */
2  OLED_ShowString(0, 0, "Threshold Set");
3
4  const char *labels[3] = {"Temp:", "Vib :", "Curr:"};
5  float values[3] = {g_threshold.temp_high, g_threshold.vib_high,
6  g_threshold.curr_high};
7  const char *units[3] = {"C", "g", "A"};
8  uint8_t y_pos[3] = {2, 4, 6};
9
10 for (int i = 0; i < 3; i++) {
11     if (g_set_mode != SET_MODE_EXIT && g_set_selected == i) {
12         /* 选中项：前面加 > 标记 */
13         OLED_ShowString(0, y_pos[i], ">");
14     }
15     OLED_ShowString(6, y_pos[i], labels[i]);
16     OLED_ShowNum(42, y_pos[i], values[i], 2, (i == 0) ? 0 : 1);
17     OLED_ShowString(78, y_pos[i], units[i]);

```

```

17 }
18
19 if (g_set_mode == SET_MODE_EXIT) {
20     OLED_ShowString(0, 7, "*: settings");
21 } else if (g_set_mode == SET_MODE_SELECT) {
22     OLED_ShowString(0, 7, "2/8:sel A:ok *:exit");
23 } else {
24     OLED_ShowString(0, 7, "2/8:+/- A:save B:back");
25 }

```

第五步：Flash 保存阈值（STM32 内部 Flash）

STM32F407 的内部 Flash 可以用来保存配置数据。用最后一个扇区（Sector 11，地址 0x080E0000，大小 128KB）存配置。

新建 `BSP/FLASH/flash_config.h` 和 `flash_config.c`。

代码块

```

1  #ifndef __FLASH_CONFIG_H
2  #define __FLASH_CONFIG_H
3
4  #include "appdata.h"
5
6  #define FLASH_CONFIG_SECTOR  FLASH_SECTOR_11
7  #define FLASH_CONFIG_ADDR     0x080E0000
8  #define FLASH_CONFIG_MAGIC    0x4D4F544F /* "MOTO" */
9
10 typedef struct {
11     uint32_t magic;
12     ThresholdConfig_t threshold;
13     uint32_t checksum;
14 } FlashConfig_t;
15
16 uint8_t FLASH_SaveThreshold(const ThresholdConfig_t *threshold);
17 uint8_t FLASH_LoadThreshold(ThresholdConfig_t *threshold);
18
19 #endif

```

代码块

```

1  #include "flash_config.h"
2  #include <string.h>
3
4  static uint32_t CalcChecksum(const uint8_t *data, uint32_t len)

```

```

5  {
6      uint32_t sum = 0;
7      for (uint32_t i = 0; i < len; i++) {
8          sum += data[i];
9      }
10     return sum;
11 }
12
13 uint8_t FLASH_SaveThreshold(const ThresholdConfig_t *threshold)
14 {
15     FlashConfig_t config;
16     config.magic = FLASH_CONFIG_MAGIC;
17     config.threshold = *threshold;
18     config.checksum = CalcChecksum((uint8_t *)&config, sizeof(FlashConfig_t) -
19 4);
20     HAL_FLASH_Unlock();
21
22     /* 擦除扇区 */
23     FLASH_EraseInitTypeDef erase_init;
24     erase_init.TypeErase = FLASH_TYPEERASE_SECTORS;
25     erase_init.Sector = FLASH_CONFIG_SECTOR;
26     erase_init.NbSectors = 1;
27     erase_init.VoltageRange = FLASH_VOLTAGE_RANGE_3;
28     uint32_t page_error = 0;
29     if (HAL_FLASHEx_Erase(&erase_init, &page_error) != HAL_OK) {
30         HAL_FLASH_Lock();
31         return 1;
32     }
33
34     /* 按字写入 */
35     uint32_t *data = (uint32_t *)&config;
36     uint32_t addr = FLASH_CONFIG_ADDR;
37     for (uint32_t i = 0; i < sizeof(FlashConfig_t) / 4; i++) {
38         if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, addr, data[i]) !=
39 HAL_OK) {
40             HAL_FLASH_Lock();
41             return 2;
42         }
43         addr += 4;
44     }
45     HAL_FLASH_Lock();
46     return 0;
47 }
48
49 uint8_t FLASH_LoadThreshold(ThresholdConfig_t *threshold)

```

```

50 {
51     FlashConfig_t *config = (FlashConfig_t *)FLASH_CONFIG_ADDR;
52
53     if (config->magic != FLASH_CONFIG_MAGIC) {
54         return 1; /* 无有效配置 */
55     }
56
57     uint32_t checksum = CalcChecksum((uint8_t *)config, sizeof(FlashConfig_t)
- 4);
58     if (checksum != config->checksum) {
59         return 2; /* 校验失败 */
60     }
61
62     *threshold = config->threshold;
63     return 0;
64 }

```

在系统初始化时加载阈值：

代码块

```

1  /* 在 main.c 或 freertos.c 的初始化代码中添加 */
2  ThresholdConfig_t loaded_threshold;
3  if (FLASH_LoadThreshold(&loaded_threshold) == 0) {
4      g_threshold = loaded_threshold;
5      printf("Threshold loaded from Flash\r\n");
6  } else {
7      printf("No valid threshold in Flash, using default\r\n");
8  }

```

 **Flash 写入注意：**(1) STM32F4 的 Flash 写入前必须先擦除，擦除按扇区进行；(2) 写入时要解锁 Flash (HAL_FLASH_Unlock)，写完后锁定；(3) 不要频繁写 Flash（寿命约 1 万次），只在用户确认保存时写；(4) 确保代码没有放在 Sector 11（检查链接脚本，一般代码从 0x08000000 开始，F407 有 1MB Flash，Sector 11 在最后 128KB，一般用不到）。

第六步：数据滤波优化

Day2 的传感器数据是原始读数，可能有抖动。添加滑动平均滤波。

新建 `APP/filter.h` 和 `filter.c`。

代码块

```

1  #ifndef __FILTER_H

```

```

2  #define __FILTER_H
3
4  #include <stdint.h>
5
6  #define FILTER_WINDOW_SIZE 10
7
8  typedef struct {
9      float buffer[FILTER_WINDOW_SIZE];
10     uint8_t index;
11     uint8_t count;
12     float sum;
13 } MovingAvgFilter_t;
14
15 void FILTER_Init(MovingAvgFilter_t *f);
16 float FILTER_Update(MovingAvgFilter_t *f, float new_value);
17 float FILTER_Get(const MovingAvgFilter_t *f);
18
19 #endif

```

代码块

```

1  #include "filter.h"
2
3  void FILTER_Init(MovingAvgFilter_t *f)
4  {
5      for (int i = 0; i < FILTER_WINDOW_SIZE; i++) {
6          f->buffer[i] = 0.0f;
7      }
8      f->index = 0;
9      f->count = 0;
10     f->sum = 0.0f;
11 }
12
13 float FILTER_Update(MovingAvgFilter_t *f, float new_value)
14 {
15     /* 减去最旧的值 */
16     if (f->count >= FILTER_WINDOW_SIZE) {
17         f->sum -= f->buffer[f->index];
18     }
19
20     /* 加入新值 */
21     f->buffer[f->index] = new_value;
22     f->sum += new_value;
23
24     f->index = (f->index + 1) % FILTER_WINDOW_SIZE;
25     if (f->count < FILTER_WINDOW_SIZE) {

```

```

26         f->count++;
27     }
28
29     return f->sum / f->count;
30 }
31
32 float FILTER_Get(const MovingAvgFilter_t *f)
33 {
34     if (f->count == 0) return 0.0f;
35     return f->sum / f->count;
36 }

```

在 SensorTask 中使用滤波：

代码块

```

1  /* 在 SensorTask 开头定义滤波器 */
2  static MovingAvgFilter_t temp_filter, vib_filter, curr_filter;
3  static uint8_t filters_initd = 0;
4
5  if (!filters_initd) {
6      FILTER_Init(&temp_filter);
7      FILTER_Init(&vib_filter);
8      FILTER_Init(&curr_filter);
9      filters_initd = 1;
10 }
11
12 /* 采集后滤波 */
13 data.temperature = FILTER_Update(&temp_filter, sht.temperature);
14 data.vibration    = FILTER_Update(&vib_filter, qmi.vibration);
15 data.current      = FILTER_Update(&curr_filter, ina.current);

```

 **滤波窗口选择：**窗口越大数据越平滑但延迟越大。温度变化慢，用 10-20 点窗口；振动变化快，用 5 点窗口；电流用 10 点窗口。上面统一用 10 点，10ms 采集周期下相当于 100ms 平均延迟，对告警响应影响可接受。

第七步：PyQt5 上位机（完整代码）

在电脑端运行，通过 MQTT 订阅数据，显示实时数值和曲线，并可远程控制继电器。

代码块

```

1  #!/usr/bin/env python3

```

```
2  # -*- coding: utf-8 -*-
3  """
4  STM32 工业电机监测系统 – PyQt5 上位机
5  依赖: pip install PyQt5 paho-mqtt pyqtgraph
6  """
7
8  import sys
9  import json
10 import time
11 from collections import deque
12
13 from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
14                               QHBoxLayout, QLabel, QPushButton, QGroupBox,
15                               QGridLayout, QComboBox, QLineEdit)
16 from PyQt5.QtCore import Qt, QTimer
17 from PyQt5.QtGui import QFont, QColor, QPalette
18
19 import paho.mqtt.client as mqtt
20 import pyqtgraph as pg
21
22 BROKER = "broker.emqx.io"
23 PORT = 1883
24 TOPIC_DATA = "motor/sensor/data"
25 TOPIC_ALARM = "motor/sensor/alarm"
26 TOPIC_CTRL = "motor/ctrl"
27
28 MAX_POINTS = 100
29
30
31 class MotorMonitorWindow(QMainWindow):
32     def __init__(self):
33         super().__init__()
34         self.setWindowTitle("STM32 工业电机监测系统 – 上位机")
35         self.setGeometry(100, 100, 1000, 700)
36
37         # 数据缓冲
38         self.times = deque(maxlen=MAX_POINTS)
39         self.temp_data = deque(maxlen=MAX_POINTS)
40         self.vib_data = deque(maxlen=MAX_POINTS)
41         self.curr_data = deque(maxlen=MAX_POINTS)
42
43         # MQTT 客户端
44         self.mqtt_client = mqtt.Client(client_id="pyqt_upper_001")
45         self.mqtt_client.on_connect = self.on_mqtt_connect
46         self.mqtt_client.on_message = self.on_mqtt_message
47         self.mqtt_connected = False
48
```

```
49         self.init_ui()
50         self.connect_mqtt()
51
52         # 定时刷新曲线
53         self.timer = QTimer()
54         self.timer.timeout.connect(self.update_plot)
55         self.timer.start(200)
56
57     def init_ui(self):
58         central = QWidget()
59         self.setCentralWidget(central)
60         main_layout = QVBoxLayout(central)
61
62         # 顶部: 连接状态 + 控制
63         top_group = QGroupBox("连接与控制")
64         top_layout = QHBoxLayout(top_group)
65
66         self.conn_label = QLabel("MQTT: 未连接")
67         self.conn_label.setStyleSheet("color: red; font-weight: bold;")
68         top_layout.addWidget(self.conn_label)
69
70         top_layout.addWidget(QLabel("Broker:"))
71         self.broker_edit = QLineEdit(BROKER)
72         self.broker_edit.setFixedWidth(150)
73         top_layout.addWidget(self.broker_edit)
74
75         self.reconnect_btn = QPushButton("重新连接")
76         self.reconnect_btn.clicked.connect(self.reconnect_mqtt)
77         top_layout.addWidget(self.reconnect_btn)
78
79         top_layout.addStretch()
80
81         self.relay_btn = QPushButton("继电器: 断开")
82         self.relay_btn.setCheckable(True)
83         self.relay_btn.clicked.connect(self.toggle_relay)
84         self.relay_btn.setStyleSheet("background-color: #4CAF50; color: white;
padding: 8px 16px;")
85         top_layout.addWidget(self.relay_btn)
86
87         main_layout.addWidget(top_group)
88
89         # 中部: 实时数值卡片
90         data_group = QGroupBox("实时数据")
91         data_layout = QGridLayout(data_group)
92
93         self.value_labels = {}
94         items = [
```

```

95         ("温度", "temp", "°C", "#2196F3"),
96         ("湿度", "humi", "%RH", "#00BCD4"),
97         ("振动", "vib", "g", "#FF9800"),
98         ("电流", "curr", "A", "#F44336"),
99         ("电压", "volt", "V", "#9C27B0"),
100        ("功率", "power", "W", "#795548"),
101    ]
102
103    for i, (name, key, unit, color) in enumerate(items):
104        row, col = divmod(i, 3)
105        card = QVBoxLayout()
106        title_label = QLabel(name)
107        title_label.setAlignment(Qt.AlignCenter)
108        title_label.setStyleSheet(f"color: {color}; font-size: 14px;")
109
110        value_label = QLabel("--")
111        value_label.setAlignment(Qt.AlignCenter)
112        value_label.setStyleSheet(f"color: {color}; font-size: 28px; font-
113weight: bold;")
114
115        unit_label = QLabel(unit)
116        unit_label.setAlignment(Qt.AlignCenter)
117        unit_label.setStyleSheet("color: #666; font-size: 12px;")
118
119        card.addWidget(title_label)
120        card.addWidget(value_label)
121        card.addWidget(unit_label)
122        data_layout.addLayout(card, row, col)
123        self.value_labels[key] = value_label
124
125        # 告警状态
126        self.alarm_label = QLabel("系统正常")
127        self.alarm_label.setAlignment(Qt.AlignCenter)
128        self.alarm_label.setStyleSheet("background-color: #4CAF50; color:
129white; "
130                                     "padding: 10px; font-size: 16px; font-
131weight: bold; "
132                                     "border-radius: 5px;")
133        data_layout.addWidget(self.alarm_label, 2, 0, 1, 3)
134
135        main_layout.addWidget(data_group)
136
137        # 底部：曲线图
138        plot_group = QGroupBox("实时曲线")
139        plot_layout = QVBoxLayout(plot_group)
140
141        self.plot_widget = pg.PlotWidget()

```

```
139         self.plot_widget.setBackground('w')
140         self.plot_widget.showGrid(x=True, y=True, alpha=0.3)
141         self.plot_widget.addLegend()
142         self.plot_widget.setLabel('left', '数值')
143         self.plot_widget.setLabel('bottom', '时间')
144
145         self.curve_temp = self.plot_widget.plot(pen=pg.mkPen('#2196F3',
width=2), name='温度(°C)')
146         self.curve_vib = self.plot_widget.plot(pen=pg.mkPen('#FF9800',
width=2), name='振动(g)')
147         self.curve_curr = self.plot_widget.plot(pen=pg.mkPen('#F44336',
width=2), name='电流(A)')
148
149         plot_layout.addWidget(self.plot_widget)
150         main_layout.addWidget(plot_group)
151
152     def connect_mqtt(self):
153         try:
154             self.mqtt_client.connect(self.broker_edit.text(), PORT, 60)
155             self.mqtt_client.loop_start()
156         except Exception as e:
157             self.conn_label.setText(f"MQTT: 连接失败 ({e})")
158
159     def reconnect_mqtt(self):
160         self.mqtt_client.loop_stop()
161         self.mqtt_client.disconnect()
162         self.connect_mqtt()
163
164     def on_mqtt_connect(self, client, userdata, flags, rc):
165         if rc == 0:
166             self.mqtt_connected = True
167             self.conn_label.setText("MQTT: 已连接")
168             self.conn_label.setStyleSheet("color: green; font-weight: bold;")
169             client.subscribe([(TOPIC_DATA, 0), (TOPIC_ALARM, 0)])
170         else:
171             self.conn_label.setText(f"MQTT: 连接失败 (rc={rc})")
172
173     def on_mqtt_message(self, client, userdata, msg):
174         try:
175             data = json.loads(msg.payload.decode())
176             if msg.topic == TOPIC_DATA:
177                 self.update_values(data)
178         except Exception as e:
179             print(f"Parse error: {e}")
180
181     def update_values(self, data):
182         now = time.strftime("%H:%M:%S")
```

```
183         self.times.append(now)
184
185         for key in ['temp', 'humi', 'vib', 'curr', 'volt', 'power']:
186             if key in data and key in self.value_labels:
187                 decimals = 1 if key in ['temp', 'humi', 'volt'] else 2
188                 self.value_labels[key].setText(f"{data[key]:.{decimals}f}")
189
190         self.temp_data.append(data.get('temp', 0))
191         self.vib_data.append(data.get('vib', 0))
192         self.curr_data.append(data.get('curr', 0))
193
194         # 告警状态
195         if data.get('alarm', 0):
196             self.alarm_label.setText("⚠ 告警中!")
197             self.alarm_label.setStyleSheet("background-color: #F44336; color:
white; "
198                                           "padding: 10px; font-size: 16px;
font-weight: bold; "
199                                           "border-radius: 5px;")
200         else:
201             self.alarm_label.setText("系统正常")
202             self.alarm_label.setStyleSheet("background-color: #4CAF50; color:
white; "
203                                           "padding: 10px; font-size: 16px;
font-weight: bold; "
204                                           "border-radius: 5px;")
205
206         def update_plot(self):
207             if len(self.times) > 0:
208                 x = list(range(len(self.times)))
209                 self.curve_temp.setData(x, list(self.temp_data))
210                 self.curve_vib.setData(x, list(self.vib_data))
211                 self.curve_curr.setData(x, list(self.curr_data))
212
213         def toggle_relay(self):
214             if self.relay_btn.isChecked():
215                 self.relay_btn.setText("继电器：吸合")
216                 self.relay_btn.setStyleSheet("background-color: #F44336; color:
white; padding: 8px 16px;")
217                 payload = json.dumps({"relay": 1})
218             else:
219                 self.relay_btn.setText("继电器：断开")
220                 self.relay_btn.setStyleSheet("background-color: #4CAF50; color:
white; padding: 8px 16px;")
221                 payload = json.dumps({"relay": 0})
222
223         if self.mqtt_connected:
```

```

224         self.mqtt_client.publish(TOPIC_CTRL, payload)
225
226     def closeEvent(self, event):
227         self.mqtt_client.loop_stop()
228         self.mqtt_client.disconnect()
229         event.accept()
230
231
232 if __name__ == "__main__":
233     app = QApplication(sys.argv)
234     app.setStyle('Fusion')
235     window = MotorMonitorWindow()
236     window.show()
237     sys.exit(app.exec_())

```

 **上位机使用方法：** (1) 电脑安装 Python 3.8+; (2) `pip install PyQt5 paho-mqtt pyqtgraph`; (3) 运行 `python motor_monitor.py`; (4) 确保电脑能访问 MQTT broker (公共 broker 需要互联网); (5) 单片机端需要在 MqttTask 中订阅 `motor/ctrl` topic 并解析继电器控制命令 (见下方补充)。

单片机端补充：解析下行控制命令

在 MqttTask 中添加对 `motor/ctrl` topic 的处理。ESP8266 收到 MQTT 消息后会通过串口输出 `+MQTTSUBRECV:0,"motor/ctrl",length,payload`，需要解析：

代码块

```

1  /* 在 esp8266.c 中添加下行消息解析 */
2  static void ESP_ParseMqttMessage(const char *line)
3  {
4      /* 格式: +MQTTSUBRECV:0,"topic",length,payload */
5      char *p = strstr(line, "+MQTTSUBRECV");
6      if (!p) return;
7
8      /* 提取 topic */
9      char *topic_start = strchr(p, '"');
10     if (!topic_start) return;
11     topic_start++;
12     char *topic_end = strchr(topic_start, '"');
13     if (!topic_end) return;
14
15     char topic[64];
16     uint16_t topic_len = topic_end - topic_start;
17     if (topic_len >= sizeof(topic)) topic_len = sizeof(topic) - 1;
18     memcpy(topic, topic_start, topic_len);

```

```

19     topic[topic_len] = '\0';
20
21     /* 提取 payload (最后一个逗号之后) */
22     char *payload_start = strchr(line, ',');
23     if (!payload_start) return;
24     payload_start++;
25
26     /* 处理控制命令 */
27     if (strcmp(topic, MQTT_TOPIC_CTRL) == 0) {
28         if (strstr(payload_start, "\"relay\":1") != NULL) {
29             RELAY_On();
30             printf("Remote: relay ON\r\n");
31         } else if (strstr(payload_start, "\"relay\":0") != NULL) {
32             RELAY_Off();
33             printf("Remote: relay OFF\r\n");
34         }
35     }
36 }

```

在 `ESP_ReadLine()` 中，每读到一行就调用 `ESP_ParseMqttMessage(line_buf)`。

常见问题排查

现象	排查方向
告警不触发	(1) 阈值设置是否太高（默认 60°C/3g/5A，手捂可能达不到）；(2) ALARM_Update 是否被 AlarmTask 正确调用；(3) 传感器数据是否正常（滤波后是否有值）
告警不恢复	(1) 迟滞比例是否太大（10% 意味着温度要降到阈值的 90% 以下）；(2) 传感器是否有偏移导致读数始终偏高；(3) 继电器是否粘住（机械继电器触点粘连）
OLED 告警闪烁不明显	(1) 闪烁周期是否合适（600ms 周期）；(2) 全屏反白是否正确实现；(3) OLED_Refresh 是否在闪烁后调用
按键设置阈值不生效	(1) 设置模式状态机是否正确切换；(2) 按键回调中 SET_MODE 判断是否正确；(3) 阈值下限保护是否阻止了修改
Flash 保存后重启丢失	(1) 写入是否成功（检查返回值）；(2) 魔术字和校验和是否正确；(3) Flash 地址是否被代码覆盖（检查链接脚本）；(4) 读取时指针是否正确
PyQt5 上位机连不上 MQTT	(1) 电脑是否能访问 broker（ping broker.emqx.io）；(2) 端口 1883 是否被防火墙拦截；(3) client_id 是否和单片机重复（必须唯一）；(4) paho-mqtt 是否正确安装

上位机收不到数据	(1) topic 是否完全一致；(2) 单片机是否在发布数据（MQTTX 验证）；(3) 上位机是否成功订阅（on_connect 中检查 rc=0）；(4) JSON 解析是否报错（看控制台输出）
远程控制继电器没反应	(1) 单片机是否订阅了 motor/ctrl topic；(2) ESP_ParseMqttMessage 是否被调用；(3) payload 格式是否匹配（"relay":1）；(4) 继电器是否被告警逻辑强制覆盖（告警时继电器强制吸合）

 **Day 5 完成标志：**用手捂住 SHT30 升温到阈值以上 → 蜂鸣器急促响、OLED 全屏闪烁告警、继电器吸合、MQTT 收到告警消息、上位机告警卡片变红；松开手温度回落到阈值 90% 以下 → 自动恢复，继电器断开，蜂鸣器停止。按键 * 进入阈值设置模式，2/8 选择和调整，A 保存到 Flash，重启后阈值不丢失。PyQt5 上位机显示六路实时数据和曲线，点击继电器按钮可远程控制。连续运行 1 小时告警功能正常。

（注：部分内容可能由 AI 生成）